Video Transcript

Video 4: Visualizing Gradient Descent Using Matplotlib (10:09)

So, we saw in gradient descent that we started with w0 and w1 at some level and we chose to be w0, 0 and w1, 0. So, that meant the slope is 0 and the Y coordinate, the intercept is 0. So, this line at the bottom here is where we started, and then we stepped and we increased w0 and we increased w1. So, w0 went to about 0.25 and the slope increased. And the target was that y = 1, was our target. So, it iterated and eventually we got the line y = 1 as our solution. Let's see, we can visualize now what's happening. So, the best model minimizes the mean square error. So, we have points and here is a Y coordinate, that's some X. X is the independent variable and our Y predicted is here.

So, we have some error and we're going to actually square the errors and take the mean. So, we're going to add them all up and then divide by the number of points and that would be our mean square error. Now, let's see if we can visualize in the error space, the error that we're making, we sometimes call that the loss function. So, in the loss function here, we have that the loss is equal to Y is the actual Y. And then X.w transpose is the prediction that we're going to make by altering the w's. So, we can see that as we alter the w's, the loss is going to get bigger or smaller. And we note that the loss is quadratic, there's a square term there. So, we're going to move around the w0 and w1 space. So, what we're going to do is visualize the loss space and here we're plotting w0 along the horizontal and w1 up.

Now, we know our solution. In this case, I'm taking the y solution to be 2 but no slope, so w1 will be 0. So, here is our solution that w0 is 2 and w1 is 0. Now we start at w0, 0 and w1, 0. So, this is where we start and then we step. So, we have a learning rate and here what's happening is we're guessing that w0 needs to increase, but w1 increases as well. Then it reaches w1, realizes that no, it now needs to go and follow this path until it reaches w0 is 2 and w1 is 0. So, let's write the code so we can actually code this up. So, the first thing we need to do is to realize that we need to plot the loss function as we vary w0 and w1. So, w0 equals 2 is where we're trying to get to. And we start at this point here, w0 equals 0 and w1 equals 0. Let's see if we can write this code. We're going to load up into tools because this will help us plot a contour. Now, first we're

© MIT xPRO® 2024. All rights reserved.

going to create a grid. So, we use line space to add a resolution. We're going to choose a resolution and we're going to divide up the space, if you like into a grid. And then, so, here's our mesh grid that we can use and we get the xy coordinates and we set the Z.

So, that's our loss to 0 here. Now we're setting our w. So, this is our goal, this is where it's going to be 0 is at (2,0) in w space. Now we call our product here, and we're going to plot as Z. We're going to plot this function that we're going to pass in, which is going to be our cost function here. So, our cost function is a lambda function in Python where we pass in w, and then it'll tell us the different X's and Y's, what the cost function is. And so, we're taking the square of the predicted 1 - y, the actual one, and we're squaring it. So, this is our cost, and we're going to plot it the cost now at every point. So, this is how we get these contours here. We've got a grid. You can see the grid in the background and we're plotting those lines. So, as we move away from the solution where w0 is 2 and w1 is 0. As we move away, we're going to get these contours. So, these w's would have all the same error on that contour. So, that represents the error if you like, and it looks like a bowl. And, here we start at 0, 0 and we go down normal to these contours. And then, we hit the plane here and it takes, there are more contours that we can't see that we're not plotting here, but that it then iterates until it gets to this point. So, here's our cost function. Now, we're going to set up the problem, and this is the same as we did before. We're going to calculate the X's with, we're going to have a row of ones, and then we're going to have all the X's.

And then for the model, we're going to calculate the Y. And here, we're doing it all with w1 equals 0, w2 equals 0. And now we say, I know, but our goal is y = y + 2. And this is where we call now the gradient descent. So, we're setting up the plot. We're going to loop over different learning rates. Here, I'm setting the learning rates as 0.01, 0.03, and 0.05. So, we're going to plot out three different learning rates. So, take a look at this code. And, here's the actual plotting. So, this is calling plot contour, that's just what we defined up here. So, we wrote it ourselves. So, that'll give us those contour lines. And now, here we're plotting the history. So, we're out of the w history, we're saying now this is the path we took in w space, and we're going to plot w0 first, and then w1. So, that gives us the red dots. You see, we're doing the red dots here, and then we do some labeling. So, let's run it. So, here we run and run again and we'll see down here our output. So, you'll see here when the learning rate is 0.01, we run out of iterations.

We get to the plane if you like, but we don't iterate to the correct solution. Now you see this

© MIT xPRO® 2024. All rights reserved.

one nearly hits the correct solution, and then with the learning rate of 0.05, we hit pretty much the correct solution. So, you can see we're going down perpendicular to these contour lines. And in fact, even here we're perpendicular to the contour lines. You can imagine other contour lines here that I'm not showing. Okay, so that's visualizing what's happening when we do gradient descent, that we're navigating some contour space. In this case, it's parabolic. We've got a square term in our loss function. In general, it'll be a more complicated space, but this allows you to understand what we're doing and how the learning rate affects the solution. Why don't you repeat this and choose different learning rates? So here, change these. Why don't you half the first one and then keep this one the same and then double the last one? So, we'll have 0.05, 0.03 and double that would be 0.1. So, why don't you run this with those different learning rates and submit the plots? Okay, bye for now.

© MIT xPRO® 2024. All rights reserved.